



Tribhuvan University

Faculty of Humanities and Social Sciences

Fake News Detection

A PROJECT REPORT

Submitted to

Department of Computer Application

Ratna Rajyalaxmi Campus

In partial fulfilment of the requirements for the Bachelors in Computer Application

Submitted by

Pashupati sah (Registration no:6-2-40-37-2020)

Under the supervisor of

Er. Shree Krishna Maharjan

November 2025



Tribhuvan University

Faculty of Humanities and Social Sciences

Ratna Rajyalaxmi Campus

Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision by **ER. SHREE KRIHNA MAHARJAN** entitled "**FAKE NEWS DETECTION**" in partial fulfilment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

Er. Shree Krishna Maharjan

Supervisor



Tribhuvan University

Faculty of Humanities and Social Sciences

Ratna Rajyalaxmi Campus

Letter of Approval

This is to certify that this project prepared by **PASHUPATI SAH** entitled “**CAR RENTAL SYSTEM**” in partial fulfilment of the requirements for the degree of Bachelor in Computer Application(BCA) has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

Signature of supervisor Er. Shree Krishna Maharjan Lecturer, project supervisor Ratna Rajyalaxmi campus	Signature of HOD Anand KC Director Department of bca Ratna rajya laxmi campus
Signature of internal examiner	Signature of external examiner

ABSTRACT

In recent years, the rapid growth of online news and social media platforms has significantly increased the spread of misleading and fabricated information, commonly referred to as fake news. This project aims to design and develop a web-based Fake News Detection System capable of identifying and classifying news articles as *real* or *fake* using machine learning techniques. The system leverages a labeled dataset of news content and applies multiple algorithms, including Logistic Regression and Multinomial Naïve Bayes, to enhance prediction accuracy. The backend is implemented in Django, while the frontend uses React for an interactive user interface. The platform features an admin panel for dataset management, model retraining, and system monitoring, as well as a user panel for real-time fake news verification. The proposed system demonstrates promising accuracy in detecting fake news and serves as a practical tool for mitigating misinformation in digital media.

Keywords: Fake news detection, Machine learning, Natural language processing, Logistic Regression, Naïve Bayes, Django, React, Misinformation, Real-time classification, Text mining.

ACKNOWLEDGEMENT

I express my sincere regard and indebtedness to my project external guide Er. Shree Krishna Maharjan , Lecturer of RatnaRajya Laxmi Campus, Kathmandu for his valuable time. I would sincerely like to thank BCA Department for giving me the opportunity to work on enhancing my technical skills while undergoing this project. This project was done under the guidance of Mr. Shree Krishna Maharjan, Lecturer of BCA Department. This project helped in understanding the various parameters which are involved in the development of a web application and the working and integration of front end along with the back end to create a fully functional web application. I would like to thank Lecturer Er. Shree Krishna Maharjan , BCA Department and whole of department for their constant support.

Table of Contents

ABSTRACT	i
ACKNOWLEDGEMENT	ii
Table of Contents	iii
LIST OF FIGURES	vi
LIST OF ABBREVIATION.....	vii
CHAPTER 1: INTRODUCTION.....	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objective.....	2
1.4 Scope And Limitation.....	3
1.4.1 Scope of the Project.....	3
1.4.2 Limitations of the Project.....	3
1.5 Development Methodology	3
1.6 Report Organization	5
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEWS	7
2.1 Background/literature reviews.....	7
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN.....	14
3.1 System Analysis.....	14
3.1.1 Requirement Analysis.....	14
I. Input News Article.....	14
II. Preprocessing of News Text	14

III. Feature Extraction	14
IV. Fake News Classification.....	14
V. Display Classification Results	14
VI. Model Evaluation (Admin/Developer).....	14
VII. User Authentication (Optional)	15
A. Software Requirements:	16
B. Hardware Requirements:	16
C. Security Requirements:	16
D. Reliability Requirements:.....	16
E. Usability Requirements:	16
F. Efficiency Requirements:.....	16
3.1.2. Feasibility Analysis	17
3.1.3. Object Modeling Using Class And Object Diagram.....	19
3.1.4. Dynamic Modeling Using State and Sequence Diagram.....	20
3.1.5. Process Modelling using Activity Diagram	22
3.2. System Design	23
3.2.1. Refinement of Class, Object, State, Sequence and Activity Diagram	23
3.2.2. Component Diagram.....	26
3.2.3. Deployment Diagram.....	27
3.3 Algorithm Details.....	27
CHAPTER 4: IMPLEMENTATION AND TESTING.....	29

4.1. Implementation.....	29
4.1.1. Tools used.....	29
4.1.2. Implement Details of Modules	30
1. Admin Module.....	30
2. Dataset Module.....	30
3. Preprocessing Module.....	31
4. Machine Learning Module	31
5. Result Storage Module.....	32
6. User Interface Module	32
4.2. Testing.....	33
4.2.1. Test Case For Unit Testing.....	33
4.2.2. Testing Cases for System Testing	34
1. Functional Testing.....	34
4.3. Result Analysis.....	35
Chapter 5: Conclusion And Future Recommendation.....	36
Conclusion	36
Future Recommendations.....	36
References	37
Bibliography	38
Appendices	38

LIST OF FIGURES

Figure 1: WaterFall Model of Fake News Detections.....	5
Figure 2: Use Case diagram of Fake News Detections	15
Figure 3: Gantt Chart of Fake News Detections	19
Figure 4: Class Diagram.....	19
Figure 5: Object Diagram Of Fake News Detection.....	20
Figure 6: Sequence Diagram Of Fake News Detection	20
Figure 7: State Diagram Of Fake News Detection.....	21
Figure 8: Activity Diagram of Fake News Detection.....	22
Figure 9: Sequence Diagram Of Fake News Detection	23
Figure 10: Object Diagram Of Fake News Detection.....	23
Figure 11: Class Diagram Of Fake News Diagram	24
Figure 12: Activity Diagram Of Fake News Detection.....	25
Figure 13:State Diagram Of Fake News Detection.....	25
Figure 14:Component Diagram Of Fake News Detection	26
Figure 15: Deployment Diagram Of Fake News Detection.....	27

LIST OF ABBREVIATION

ML	Machine Learning
LR	Logistic Regression
NB	Naïve Bayes
RF	Random Forest
TF-IDF	Term Frequency – Inverse Document Frequency
UI	User Interface
DB	Database
SQL	Structured Query Language
CSV	Comma-Separated Values
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
F1-Score	Harmonic mean of Precision and Recall
API	Application Programming Interface
SQLite3	Lightweight relational database system
BERT	Bidirectional Encoder Representations from Transformers
IDE	Integrated Development Environment

CHAPTER 1: INTRODUCTION

1.1 Introduction

In the digital era, news and information have become more accessible than ever before. The rapid growth of the internet and social media platforms has revolutionized the way people consume news, making it possible to access updates in real-time from anywhere in the world. However, this convenience has also given rise to a significant problem: the spread of misinformation and fake news. Fake news can be defined as deliberately fabricated information intended to mislead readers or influence public opinion. The consequences of such misinformation can range from creating public panic to influencing political outcomes, making the detection of fake news a crucial task.

With the increasing use of social media platforms like Facebook, Twitter, and Instagram, fake news spreads faster and reaches a larger audience compared to traditional media. The lack of a reliable verification mechanism in these platforms allows unverified content to be shared freely, often without fact-checking. As a result, individuals and organizations are facing challenges in distinguishing between legitimate news and false information. This has created an urgent need for intelligent systems that can automatically detect and classify fake news with high accuracy.

Machine Learning (ML) and Natural Language Processing (NLP) have emerged as powerful tools to address this challenge. By analyzing large datasets of labeled news articles, ML models can learn patterns and linguistic features that differentiate fake news from real news. In this project, two algorithms — **Logistic Regression** and **Multinomial Naïve Bayes** — are used to train local models for fake news detection. Both algorithms are implemented and trained using a pre-collected dataset stored locally, without the use of any external APIs. This ensures data privacy, offline functionality, and full control over the training and prediction process.

The primary objective of this project is to design and develop a standalone, web-based Fake News Detection System that combines the results of these two algorithms to enhance detection accuracy. The backend is implemented in Django for model integration and data processing, while the frontend is developed using React for a clean and interactive user interface. The system features an admin panel for dataset management, model retraining, and monitoring, as well as a user panel that allows individuals to input news content and instantly receive classification results indicating whether the news is real or fake.

By implementing this project, we aim to contribute to the ongoing global effort to minimize the harmful effects of misinformation. The system provides a reliable, independent, and secure platform for detecting fake news without relying on third-party services. Future improvements may include the integration of deep learning models, multilingual datasets, and continuous retraining to adapt to evolving patterns of misinformation. Ultimately, the Fake News Detection System promotes responsible information sharing and helps preserve the credibility of digital media.

1.2 Problem Statement

- **Rapid Spread of Misinformation:** The widespread use of social media and online platforms has made it easier for fake news to reach large audiences quickly, causing confusion, panic, and potential harm.
- **Difficulty in Manual Verification:** Manually fact-checking news content is time-consuming, labor-intensive, and impractical for large volumes of data, leading to delayed or missed detection of fake news.
- **Lack of Automated, Offline Solutions:** Many fake news detection tools depend on third-party APIs or online services, which limits their availability and privacy; there is a need for a fully offline, locally trained system for accurate and secure detection.

1.3 Objective

- **Develop an Online Detection System:** Build a web-based platform accessible via browser that detects fake news using locally stored datasets and trained models, without relying on external APIs.
- **Implement Multiple Machine Learning Algorithms:** Train and evaluate Logistic Regression, Naïve Bayes, and Random Forest models on the backend to improve prediction accuracy and reliability.
- **Provide an Interactive User Interface:** Design a user-friendly panel for instant news verification and an admin panel for managing datasets, retraining models, and monitoring system performance.

1.4 Scope And Limitation

1.4.1 Scope of the Project

- **Online Accessibility:** The system is web-based and can be accessed through a browser, allowing users to check news articles for authenticity anytime.
- **Machine Learning-Based Detection:** Utilizes multiple locally trained algorithms — Logistic Regression, Naïve Bayes, and Random Forest — to classify news as real or fake.
- **Admin and User Panels:** Provides an admin panel for dataset management, model retraining, and monitoring, and a user panel for instant fake news verification.
- **Dataset-Driven System:** Operates entirely on pre-collected datasets, ensuring consistent and controlled training for reliable predictions.

1.4.2 Limitations of the Project

- **Dataset Dependency:** The accuracy of the system depends on the quality and size of the training dataset; unseen or novel news patterns may reduce prediction accuracy.
- **Language Restriction:** The system is primarily designed for a specific language (e.g., English or Nepali) and may not accurately detect fake news in other languages.
- **No Real-Time Web Scraping:** The system does not fetch or verify news directly from external websites or APIs; users must manually input news content for verification.
- **Limited Advanced NLP Features:** While effective for text classification, it does not currently implement advanced techniques like deep learning transformers or semantic analysis for highly nuanced content.

1.5 Development Methodology

- **Requirement Analysis:** The first step involved gathering requirements for the system, including identifying the key features such as news input, real-time classification, admin management, and the use of multiple machine learning algorithms. The scope, objectives, and constraints of the project were clearly defined to guide the development process.
- **System Design:** In this phase, the overall architecture of the system was designed. The backend was planned using Django to handle data processing, model training, and prediction logic, while React was chosen for the frontend to create a responsive and

interactive user interface. Database schemas for storing news data, model parameters, and user inputs were also designed.

- **Data Collection and Preprocessing:** A labeled dataset of news articles was collected and organized for model training. Data preprocessing steps included text cleaning, tokenization, removal of stop words, and feature extraction using techniques like TF-IDF. These steps ensure that the data is suitable for training machine learning models accurately.
- **Model Development and Training:** Three machine learning algorithms — Logistic Regression, Naïve Bayes, and Random Forest — were implemented and trained using the preprocessed dataset. The models were evaluated for accuracy, precision, and recall to determine their effectiveness in detecting fake news. The best-performing model(s) were integrated into the system for live predictions.
- **System Implementation:** The frontend and backend were integrated to create a functional web application. The admin panel allows for dataset management, model retraining, and performance monitoring, while the user panel provides an interface for users to input news content and receive instant classification results.
- **Testing and Evaluation:** The system underwent rigorous testing, including unit testing, integration testing, and user acceptance testing. Model performance was assessed using metrics such as accuracy, precision, recall, and F1-score. Feedback from test users was used to refine the interface and functionality.
- **Deployment and Maintenance:** Finally, the system was deployed on a local or web server to make it accessible online. Periodic maintenance and updates, such as adding new dataset entries or retraining models, ensure the system remains effective over time.
- **Conclusion:**
The chosen methodology ensures a structured approach to development, from initial analysis to deployment, and emphasizes both technical accuracy in model training and user accessibility in the web interface. This approach allows the project to deliver a reliable and practical solution for fake news detection.

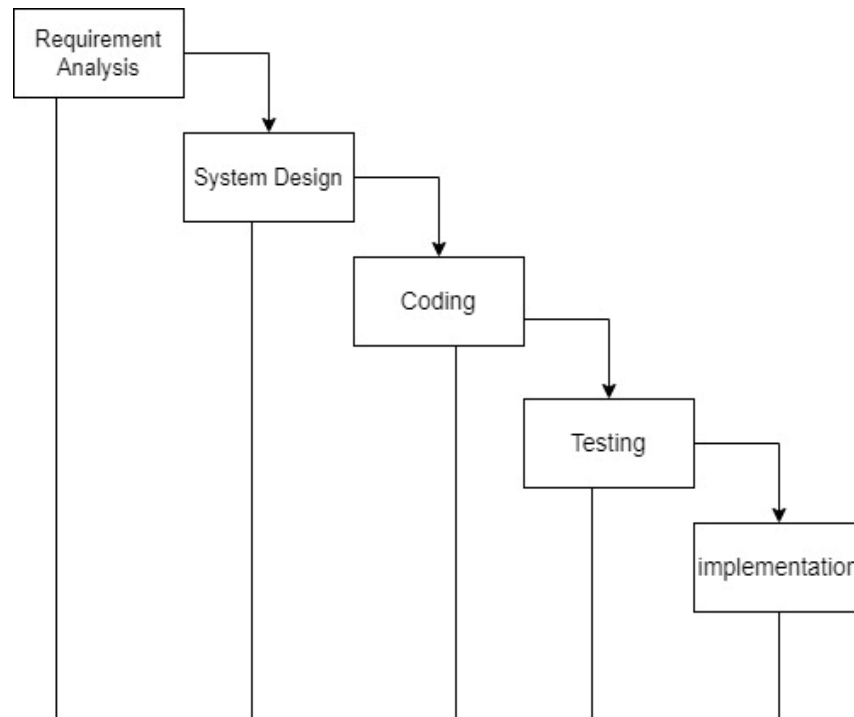


Figure 1: WaterFall Model of Fake News Detections

1.6 Report Organization

- **Chapter 1 – Introduction:** Provides an overview of the project, including the background, significance, objectives, and scope of the system. It highlights the problem of fake news and the need for an effective detection solution.
- **Chapter 2 – Literature Review / Background:** Discusses related works, previous research, and existing methods in fake news detection. This chapter also explains the theoretical foundations of machine learning algorithms used in the project, such as Logistic Regression, Naïve Bayes, and Random Forest.
- **Chapter 3 – Development Methodology:** Describes the step-by-step methodology followed in the development of the system, including requirement analysis, system design, data preprocessing, model training, system implementation, and testing.
- **Chapter 4 – System Design and Architecture:** Explains the overall system architecture, data flow, database design, and interaction between the frontend, backend, and machine learning models. It may include diagrams such as flowcharts and block diagrams for clarity.

- **Chapter 5 – Implementation and Results:** Details the implementation process, including frontend and backend integration, user interface design, and machine learning model deployment. It also presents the results of testing and model evaluation with accuracy metrics.
- **Chapter 6 – Conclusion and Future Work:** Summarizes the achievements of the project, highlights its limitations, and suggests possible improvements or extensions for future development.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEWS

2.1 Background/literature reviews

A thorough review of recent research papers, articles, and case studies was conducted to understand the strengths and weaknesses of various fake news detection approaches. Studies highlight that supervised learning algorithms like Logistic Regression, Naive Bayes, and Random Forest are effective for text classification tasks, especially when combined with efficient preprocessing and feature extraction techniques such as TF-IDF. Additionally, ensemble methods like Random Forest help improve accuracy by reducing overfitting.

Fake news has become a growing concern in the digital age, particularly on social media platforms. Researchers have explored various approaches to automatically detect and filter fake news. One of the most widely used techniques is supervised machine learning, where models are trained on labeled datasets to classify news as real or fake.

Allcott and Gentzkow (2017) highlighted the role of social media in spreading misinformation during the 2016 U.S. election. Their study showed the urgent need for detection systems that can scale to large volumes of data. Shu et al. (2017) provided a comprehensive review of detection techniques, emphasizing content-based and context-based analysis.

Naive Bayes, Logistic Regression, and Random Forest are among the commonly used algorithms for text classification in fake news detection. These models work well with structured input data and are efficient to train. Feature extraction methods like Term Frequency-Inverse Document Frequency (TF-IDF) enhance the accuracy of these models by highlighting important words in the news content.

Devlin et al. (2019) introduced BERT, a deep learning model that improved performance in natural language processing tasks. BERT can understand the context of words better than traditional models, making it more accurate but also more resource-intensive.

Pérez-Rosas et al. (2018) developed a dataset of real and fake news to test machine learning models. Their experiments showed that with proper preprocessing and feature extraction, simple

algorithms like Logistic Regression could achieve good accuracy. Thota et al. (2018) suggested that ensemble models like Random Forest further improve performance by combining the results of multiple classifiers.

Wang (2017) introduced the LIAR dataset, which includes short political statements and metadata for fake news detection. The dataset has been widely used to benchmark the performance of various models.

Rubin et al. (2016) pointed out the difficulty in distinguishing fake news from satire. This shows the importance of understanding the context and tone of content while designing fake news classifiers.

Arrieta et al. (2020) reviewed explainable AI approaches, emphasizing that detection systems should not only be accurate but also interpretable to users. Ethical concerns were also addressed by Bhattacharya and Arora (2023), who emphasized the importance of fairness and transparency in fake news detection systems.

Shao et al. (2018) developed Hoaxy, a tool to visualize the spread of misinformation. Their work supports the idea that combining content analysis with social network analysis improves detection performance.

In summary, literature suggests that a combination of text preprocessing, feature extraction, and machine learning algorithms can be effective for fake news detection. Simple models like Naive Bayes and Logistic Regression are easy to implement, while ensemble methods like Random Forest provide better accuracy. The integration of ethical and explainable AI is also crucial for building trustworthy systems.

Further research by Conroy et al. (2015) discussed automated deception detection and explored both linguistic and network-based features. Their work indicates that combining multiple data sources and analysis types leads to better accuracy in detecting misinformation. This inspired many follow-up studies to look beyond just the text of news articles and consider their social sharing behavior. The European Commission (2018) released a Code of Practice on Disinformation, which encouraged platforms and researchers to collaborate in tackling the fake news problem. Such

policies stress the growing importance of automatic fake news detection tools in the real world, not just in academic research.

One major challenge highlighted in the literature is the subjectivity in labeling news as fake or real. Some news stories are not entirely false but are misleading or biased. This raises the need for more advanced models that can understand subtle variations in tone, bias, and framing.

Bhattacharya and Arora (2023) explored ethical concerns in AI-based fake news detection. They emphasized that the models must be designed with fairness and privacy in mind. It's important that these systems do not wrongly classify news based on author bias, political leaning, or unreliable training data.

Recent advancements in Natural Language Processing (NLP) have enabled the use of word embeddings like Word2Vec, GloVe, and transformer-based models. These allow systems to capture semantic meanings and improve classification. However, they also increase the computational load and complexity of the system.

Another interesting approach discussed by researchers is the use of hybrid systems that combine machine learning and rule-based models. These systems aim to benefit from the flexibility of machine learning and the accuracy of handcrafted rules.

Explainable AI (XAI) techniques have become important in this domain. Users need to know why a piece of news is marked as fake. Arrieta et al. (2020) suggested that transparency in model predictions builds trust and allows easier debugging and improvements in the system.

Real-time fake news detection is still a challenge. Most models work in batch mode and require time to process data. Future studies suggest integrating such systems into browser plugins, social platforms, and mobile apps to flag misinformation in real-time. Researchers also explored the importance of datasets in model training. Balanced datasets with equal amounts of real and fake news improve performance. Datasets like LIAR, Kaggle and FakeNewsNet have helped standardize evaluation across studies.

Finally, it is evident from the literature that no single model performs best in all cases. Different algorithms have strengths and weaknesses depending on the dataset, language, and features used. Therefore, many studies suggest comparing multiple algorithms and selecting the one that performs best for the specific task at hand.

Another key insight from recent research is the role of preprocessing techniques. Most studies agree that removing stop words, punctuation, and special characters from the text improves the performance of machine learning models. Some researchers also suggest lemmatization and stemming to reduce word forms and improve feature consistency.

Studies also show that vectorization techniques like TF-IDF and CountVectorizer are crucial for converting text into numerical data that algorithms can understand. These methods assign importance to words based on how frequently they appear, improving the model's ability to distinguish between fake and real news.

Recent approaches have also used sentiment analysis to identify fake news. The idea is that fake news often carries emotional language or strong bias. By analyzing sentiment, models can catch manipulative or exaggerated content, which may indicate misinformation.

Research by Thota et al. (2018) demonstrated the use of ensemble models, where the outputs of multiple classifiers are combined to make a final decision. This reduces the risk of errors caused by a single algorithm and boosts overall accuracy. Random Forest is a widely used ensemble technique in this space.

Deep learning models, such as Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs), have also been explored. These models can learn complex patterns in text but require more data and computational resources. While powerful, they are less interpretable compared to traditional models.

Shu et al. (2017) suggested that fake news detection is more effective when combining content-based and context-based features. This means analyzing not only the news text but also how it is shared on social media—such as who shares it, how quickly it spreads, and which platforms it appears on.

Graph-based models are another promising direction. These models map how fake news spreads across users and platforms, identifying patterns in the sharing behavior. This helps detect coordinated misinformation campaigns that may not be obvious from text alone.

Moreover, some researchers have tried incorporating metadata, such as publication date, author name, and source credibility, into the model. This auxiliary information adds value and helps the system make more informed decisions.

Another growing area is cross-lingual fake news detection, where models are trained to detect misinformation in multiple languages. This is crucial in multilingual countries like India and Nepal, where fake news often spreads in regional languages.

Finally, there is a strong call in the literature for the development of user-friendly tools that allow end-users to check the authenticity of news. These tools could be web-based or mobile applications that use the backend ML model to deliver real-time predictions.

Another interesting area of research focuses on source verification. Some fake news detection systems begin by checking the credibility of the source or website. If the source is known for publishing misinformation, the article is flagged more quickly. This approach is effective in reducing processing time.

Temporal analysis has also been introduced in some studies. Researchers found that the timing of how news spreads — including when it's published and how fast it gets shared — can help determine its authenticity. Fake news often spreads faster than verified news, especially on social platforms.

A few papers discussed the importance of using human-in-the-loop systems, where machine learning models assist human fact-checkers by prioritizing suspicious articles. These systems improve overall accuracy because human judgment is used to correct or verify machine predictions.

Recent advancements in transformer models like BERT (Bidirectional Encoder Representations from Transformers) have significantly improved the accuracy of fake news detection. These

models understand context better than older NLP models and can detect fake news with higher precision.

The concept of zero-shot learning has also been proposed, where models are trained to detect fake news without needing many examples of fake and real news. This is especially useful in emerging situations where labeled data is limited.

Data labeling challenges are also well-documented. Several studies report that inconsistent or biased labeling by human annotators reduces model performance. Therefore, quality datasets with unbiased, consistent labels are crucial for effective model training.

Some research highlights the risk of model overfitting, especially when using deep learning on small datasets. Overfitting occurs when the model performs well on training data but poorly on new, unseen data. Using regularization techniques and cross-validation helps address this issue.

In terms of user impact, researchers agree that detection systems should not only identify fake news but also explain the reasons behind it. Explainability improves user trust and allows better adoption of the system in educational and professional environments.

The role of fact-checking organizations has been emphasized in the literature. These organizations can provide verified data to train models. Collaborating with them ensures that ML systems are aligned with real-world standards.

Lastly, literature suggests that continuous learning systems that update their models based on new data are more effective. Fake news evolves in style and content over time, so models must adapt regularly to stay effective.

Another notable aspect discussed in many research papers is the importance of using balanced datasets. Since real news is usually more abundant than fake news, imbalanced datasets can cause models to become biased. Researchers recommend techniques like oversampling, undersampling, or synthetic data generation (like SMOTE) to balance the classes.

Some literature also emphasizes the use of topic modeling techniques, such as Latent Dirichlet Allocation (LDA), to identify hidden themes in news articles. These topics can help distinguish fake news, which often focuses on sensational or emotionally charged topics.

The impact of language style and complexity has been explored. Studies show that fake news tends to use simpler language, exaggerated claims, and informal tones compared to legitimate news. These language cues can be captured using textual analysis and used as features for classification.

Research also shows the usefulness of lexical features such as word count, sentence length, and frequency of certain keywords. These surface-level features may seem basic but often provide strong indicators of whether the content is fabricated.

In some cases, researchers have tried combining textual data with image data, especially in social media fake news posts that contain memes or photos. Though more complex, multi-modal approaches can detect fake news more effectively by evaluating both text and visual cues.

A few researchers have investigated cross-domain fake news detection, where models trained on one topic (e.g., politics) are used to detect fake news in another (e.g., health). This helps in building more generalized models that are not limited to one content domain.

Studies also highlight the role of psychological and emotional manipulation in fake news. Fake stories often aim to trigger anger, fear, or excitement. By analyzing emotional tone or polarity in the content, systems can better detect manipulative articles. Another stream of research deals with real-time fake news detection. These studies stress the need for quick predictions, especially during elections or crises. Lightweight models like Logistic Regression and Naive Bayes are often used for faster processing.

Many papers also recommend using model interpretability tools such as LIME or SHAP to explain predictions. This transparency helps users trust the system, especially in sensitive domains like journalism or education. Finally, some recent works have experimented with hybrid models that combine rule-based systems with machine learning. This blend brings together human expertise (rules) and automated learning (ML), producing more accurate and reliable fake news detection systems.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

3.1.1 Requirement Analysis

A) Functional Requirement

I. Input News Article

- The user must be able to input or upload a news article text for analysis.

II. Preprocessing of News Text

- The system automatically cleans and preprocesses the input text (removes stop words, punctuation, tokenizes).

III. Feature Extraction

- The system converts the cleaned text into numerical features using TF-IDF vectorization.

IV. Fake News Classification

- The system runs the news text through multiple machine learning models (Logistic Regression, Naive Bayes, Random Forest) for classification.

V. Display Classification Results

- The system shows the user a clear output indicating if the news is **Real** or **Fake** along with confidence scores.

VI. Model Evaluation (Admin/Developer)

- The system allows viewing performance metrics (accuracy, precision, recall, F1-score) for model evaluation and improvement.

VII. User Authentication (Optional)

Users can optionally sign up and log in to access extended features like saving past results or feedback.

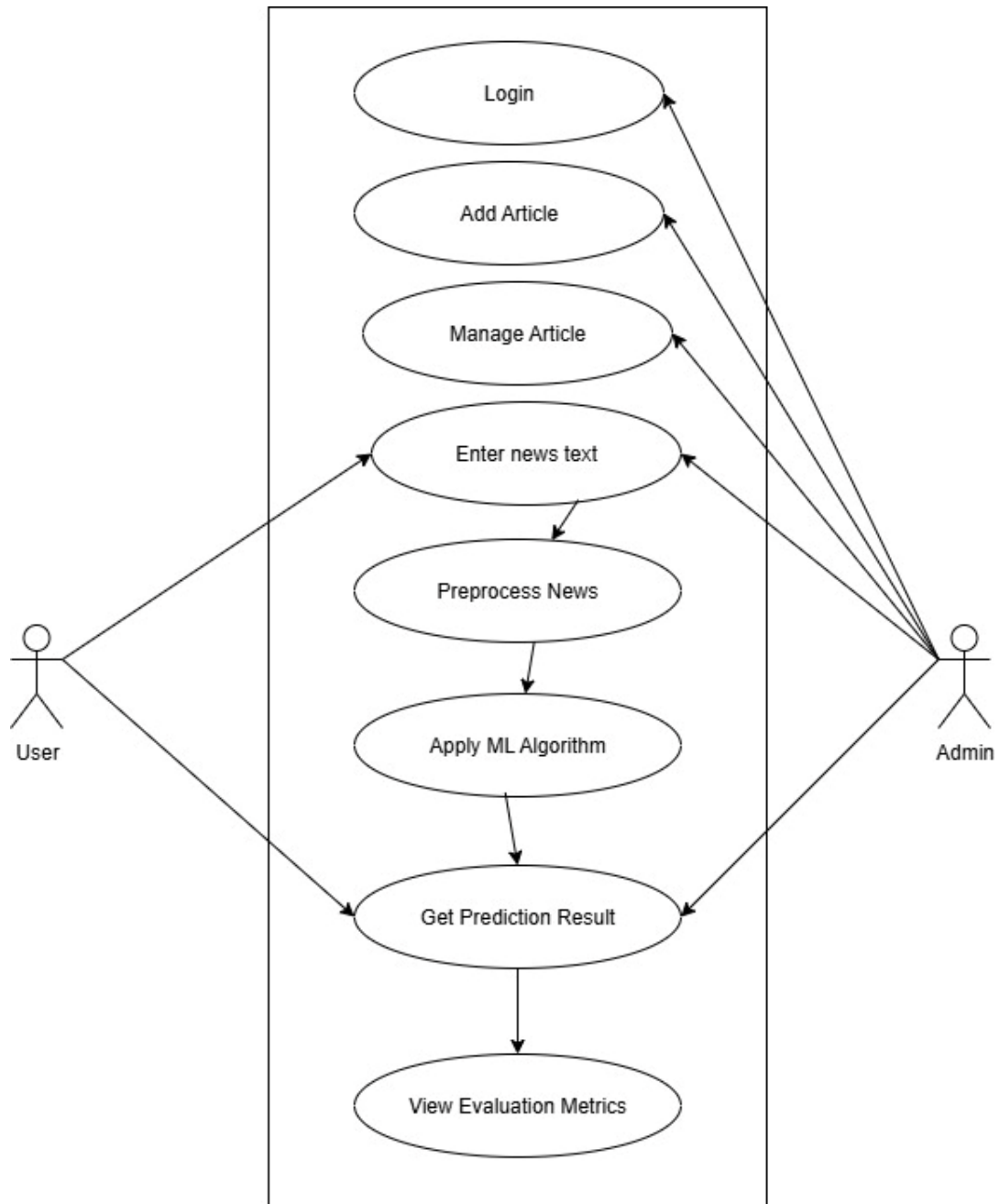


Figure 2: Use Case diagram of Fake News Detections

B) Non-Functional Requirement

A. Software Requirements:

- Python programming language
- Jupyter Notebook or any Python IDE
- Libraries: scikit-learn, NLTK, pandas, NumPy, matplotlib
- Optional: Flask or Django (for web interface)

B. Hardware Requirements:

- Minimum 4 GB RAM (recommended for smooth model training)
- At least 10 GB free storage for datasets and models

C. Security Requirements:

- The system should ensure the privacy and security of user-submitted news text.
- Data must not be exposed to unauthorized users during processing and storage.

D. Reliability Requirements:

- The system should provide accurate and consistent classification results.
- The system must handle unexpected inputs without crashing.

E. Usability Requirements:

- The user interface should be simple and intuitive for users to submit news articles easily.
- Clear display of results indicating whether news is fake or real.

F. Efficiency Requirements:

- The system should provide classification results within a reasonable response time.
- It should efficiently handle multiple requests if deployed as a web service.
- Performance metrics (accuracy, precision, recall, F1-score) for model evaluation and improvement.

3.1.2. Feasibility Analysis

1. Technical Feasibility:

Technical feasibility assesses whether the project can be developed with the available technology and resources. The Fake News Detection System is technically feasible because:

- The backend is implemented using **Django**, which supports integration of machine learning models and database management.
- The frontend is developed using **React**, providing a responsive and interactive interface for users.
- Machine learning models, including **Logistic Regression, Naïve Bayes, and Random Forest**, can be trained and deployed locally on the server without external dependencies.
- Required hardware and software resources (like a computer with sufficient memory, Python, libraries like scikit-learn, pandas, and NumPy) are available.
- No external APIs are needed, which simplifies technical requirements and ensures data control.

2. Operational Feasibility:

Operational feasibility examines whether the system will function effectively in the intended environment and satisfy user needs. This project is operationally feasible because:

- Users can access the system online through a web browser to verify news instantly.
- The **user panel** provides a simple interface for inputting news content, while the **admin panel** allows managing datasets, retraining models, and monitoring performance.
- The system is designed to handle a realistic volume of news submissions, with scalability for future expansion.
- It addresses the operational need to reduce misinformation and provides a practical solution for everyday users and administrators.

3. Economic Feasibility:

Economic feasibility assesses whether the project is cost-effective and financially viable. This project is economically feasible because:

- Open-source technologies like Django, React, Python, and ML libraries are used, minimizing software costs.

- Hardware requirements are modest; a standard server or cloud hosting with sufficient memory and storage is enough.
- The project does not require paid APIs or external services, reducing operational costs.
- The benefits of reducing fake news, providing a reliable tool for verification, and improving digital literacy outweigh the development and maintenance costs.

4. Schedule Feasibility:

Schedule feasibility evaluates whether the project can be completed within a reasonable timeframe.

This project is schedule-feasible because:

- The development plan follows a structured methodology, starting with requirement analysis, design, data collection, preprocessing, model training, implementation, testing, and deployment.
- Each phase has a clearly defined timeline, allowing completion within the semester timeframe.
- The project scope is well-defined, focusing on a core functionality of detecting fake news using three ML algorithms without overly complex integrations.

Month	Jesth 2082				Ashad 2082				Shrawan 2082			
Work week	Week 1	Week 2	Week 3	Week	Week	Week	Week 3	Week	Week	Week	Week	Week
Project selection												
proposal preparation												
Planning & Analysis												
Design												
Coding												
Testing												
Implementation												
Documentation												

Figure 3: Gantt Chart of Fake News Detections

3.1.3. Object Modeling Using Class And Object Diagram

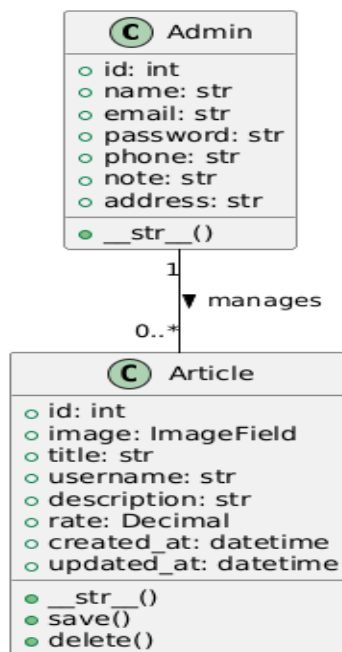


Figure 4: Class Diagram

The class diagram shows two main classes: Admin and Article. An Admin can manage multiple Article instances (one-to-many relationship). Each class lists key attributes and main methods (save, delete) reflecting the Django model functionality.

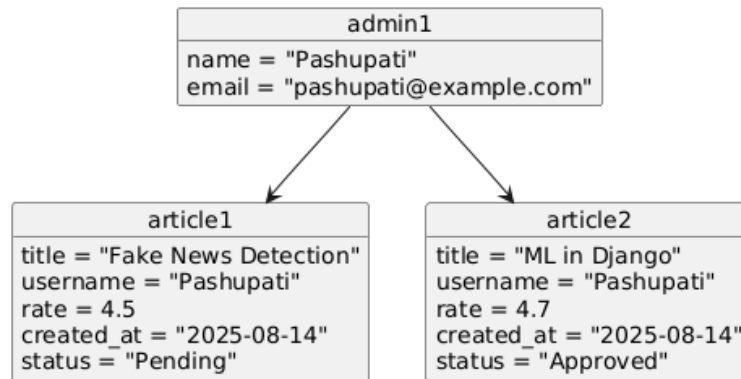


Figure 5: Object Diagram Of Fake News Detection

The object diagram shows a snapshot of system instances at runtime. admin is an instance of Admin managing two Article objects (article1 and article2). This diagram represents real data flow and object relationships at a specific point in time.

3.1.4. Dynamic Modeling Using State and Sequence Diagram

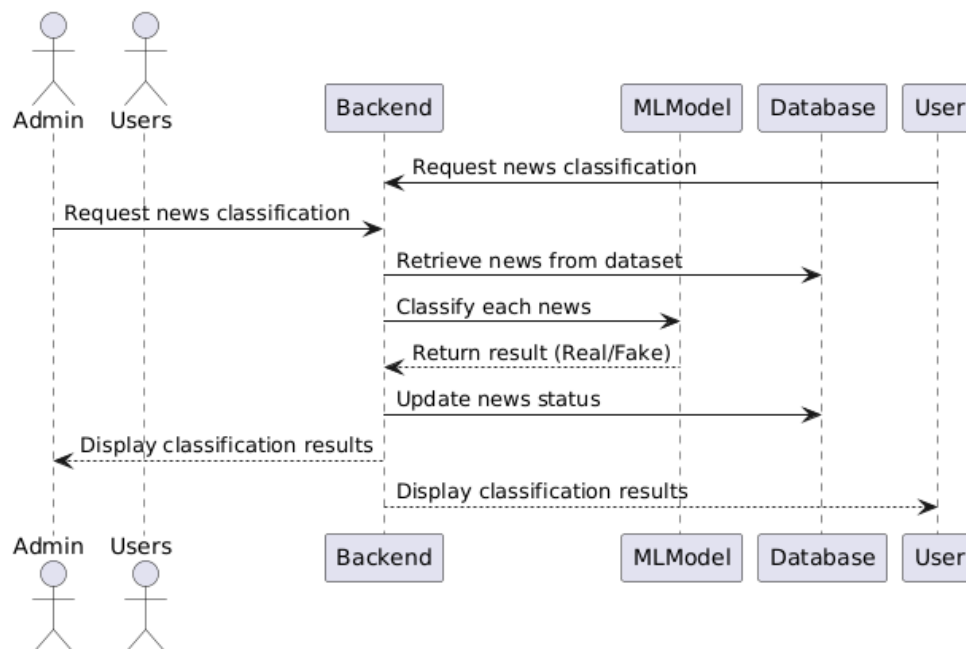


Figure 6: Sequence Diagram Of Fake News Detection

- The **Admin/Users** triggers the classification process.
- The **Backend** retrieves news articles from the **Database** (dataset).
- Each article is classified by the **MLModel**.
- Classification results are updated in the database and sent to the **Admin** for viewing.

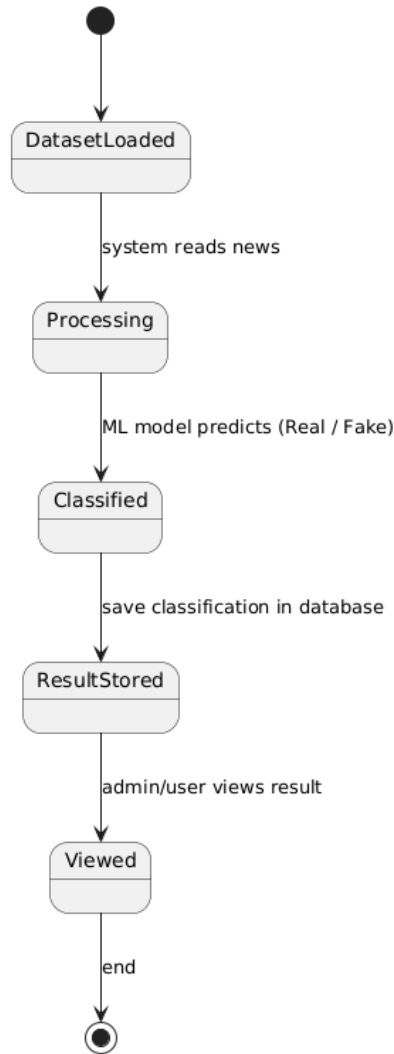


Figure 7: State Diagram Of Fake News Detection

- The news article starts in **DatasetLoaded** state.
- The system processes each article using the ML models (**Processing**).
- After prediction, the article reaches the **Classified** state.
- The result is stored in the database (**ResultStored**) and can be viewed by the admin or user (**Viewed**).

3.1.5. Process Modelling using Activity Diagram

- The process starts by **loading the dataset** from the database.
- Articles are **preprocessed** for model input (cleaning, tokenization, feature extraction).
- The system selects the **ML model** for classification.
- Each news article is **classified** as Real or Fake.
- Once classification is complete, the results are **stored in the database**.
- Finally, results are **displayed to the admin or user**.

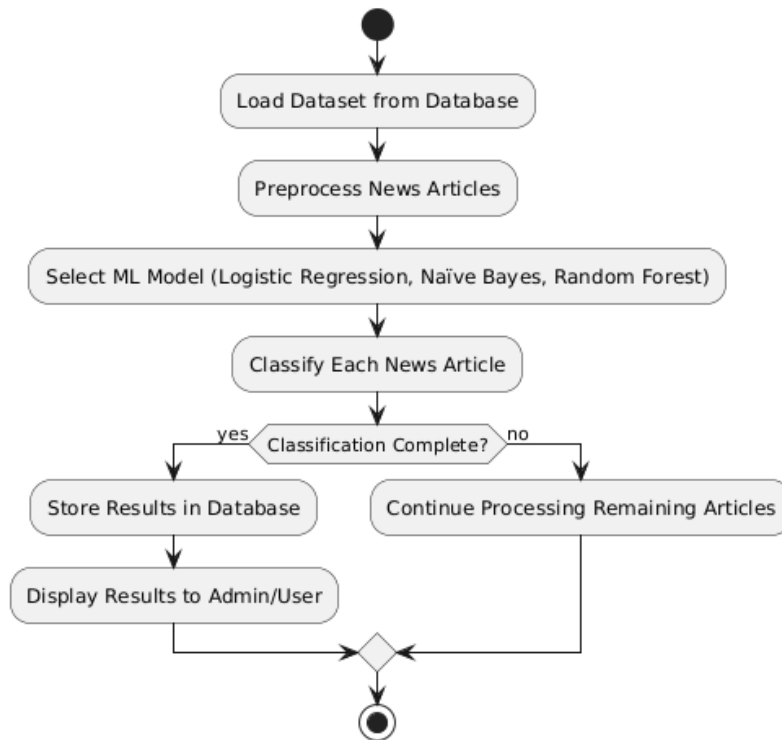


Figure 8: Activity Diagram of Fake News Detection

3.2. System Design

3.2.1. Refinement of Class, Object, State, Sequence and Activity Diagram

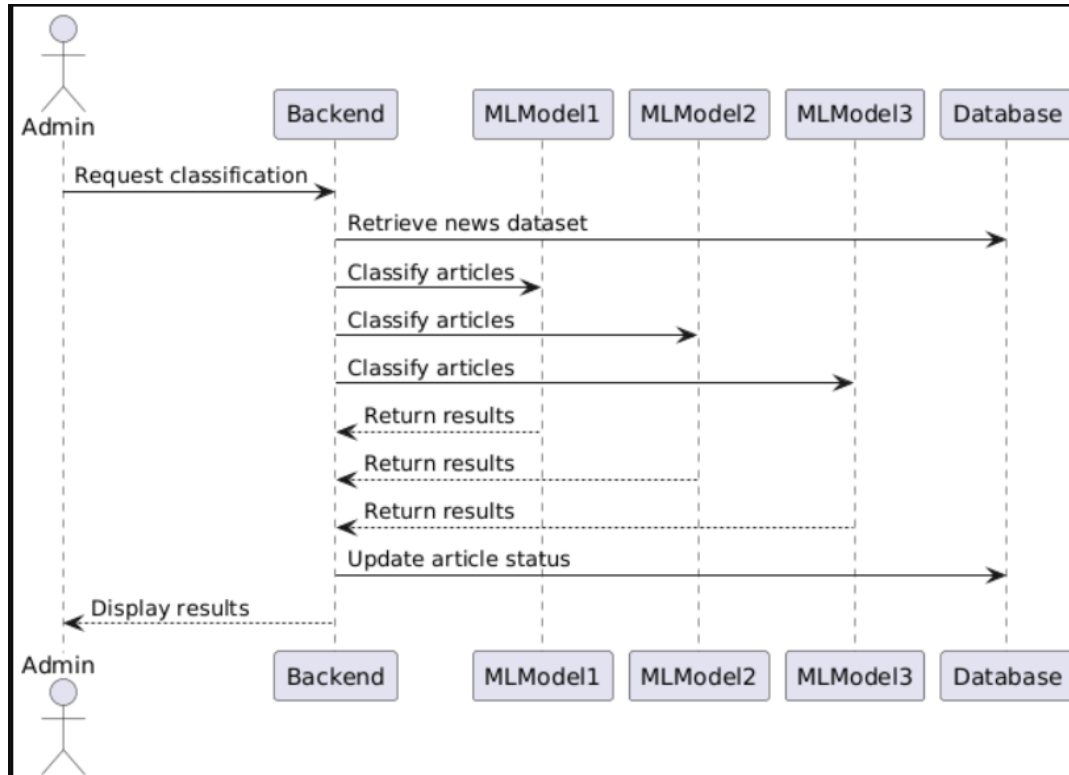


Figure 9: Sequence Diagram Of Fake News Detection

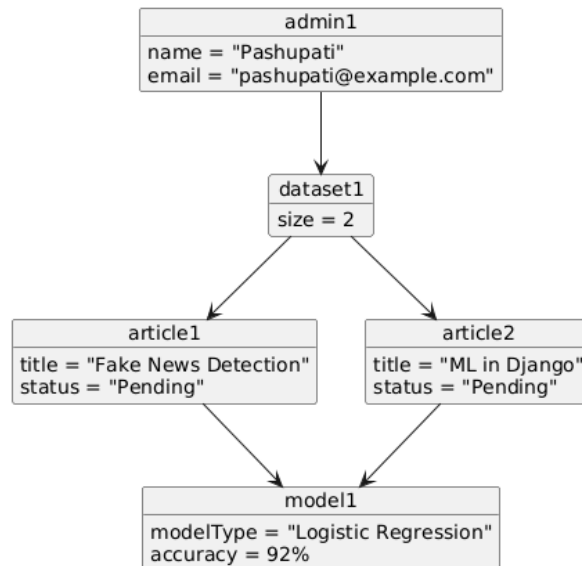


Figure 10: Object Diagram Of Fake News Detection

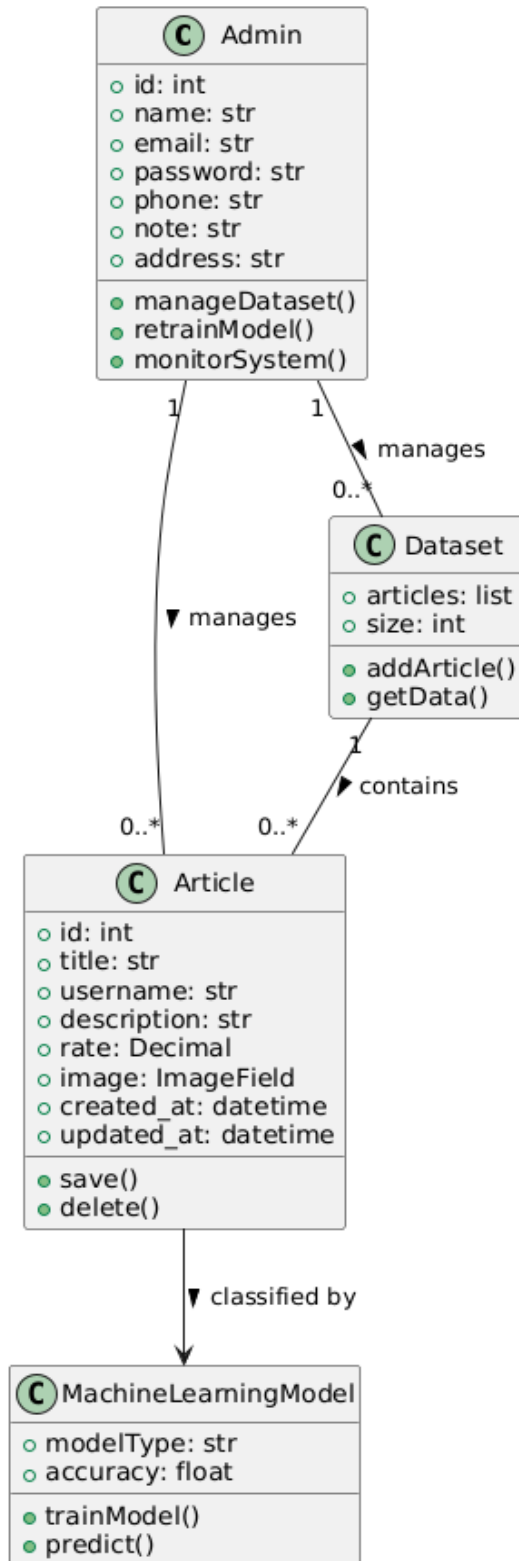


Figure 11: Class Diagram Of Fake News Diagram

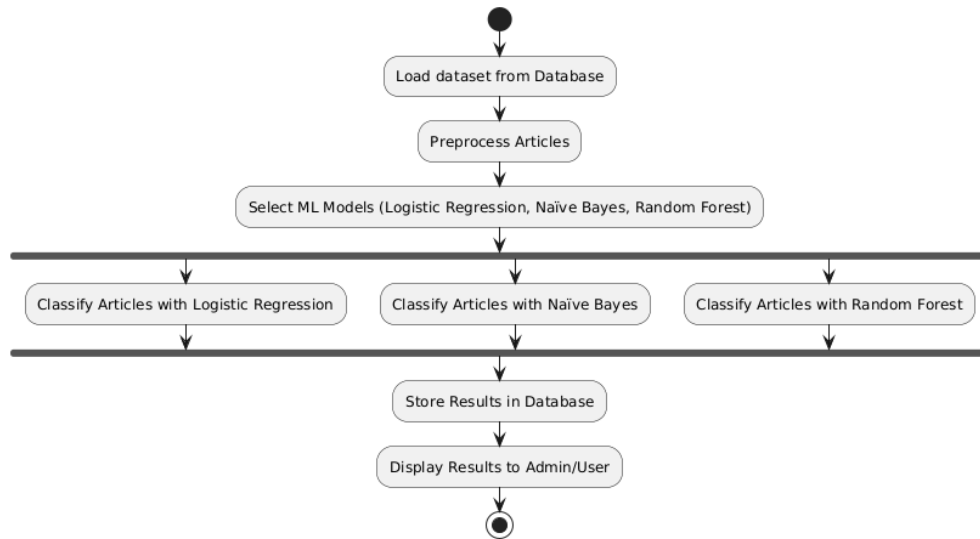


Figure 12: Activity Diagram Of Fake News Detection

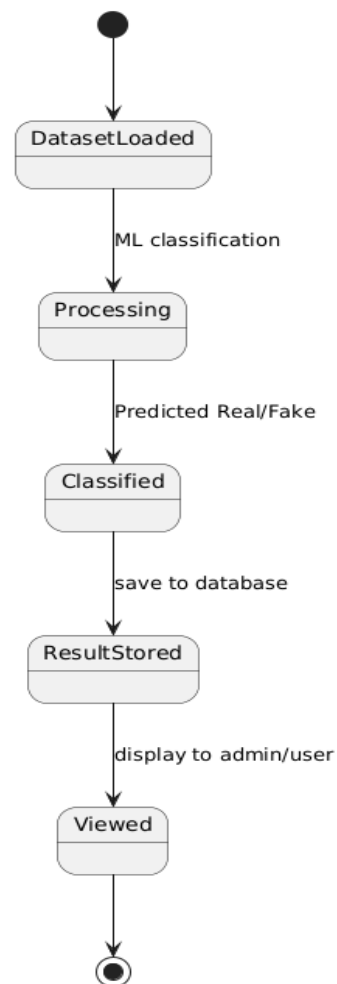


Figure 13:State Diagram Of Fake News Detection

3.2.2. Component Diagram

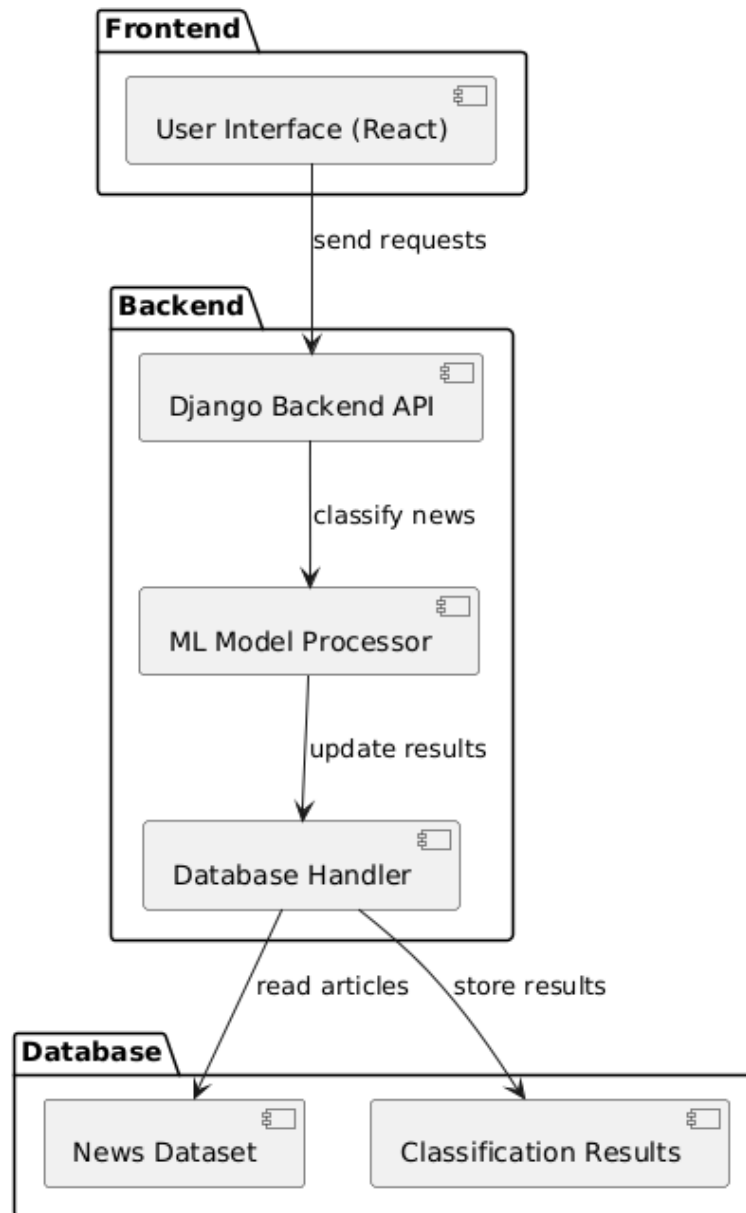


Figure 14: Component Diagram Of Fake News Detection

- **Frontend:** The React-based user interface handles user/admin interaction.
- **Backend:** Django handles requests, processes news classification through ML models, and interacts with the database.
- **Database:** Stores the pre-collected news dataset and classification results.
- **Flow:** User/Admin interacts with the frontend → backend retrieves dataset → ML models classify news → results are stored and displayed.

3.2.3. Deployment Diagram

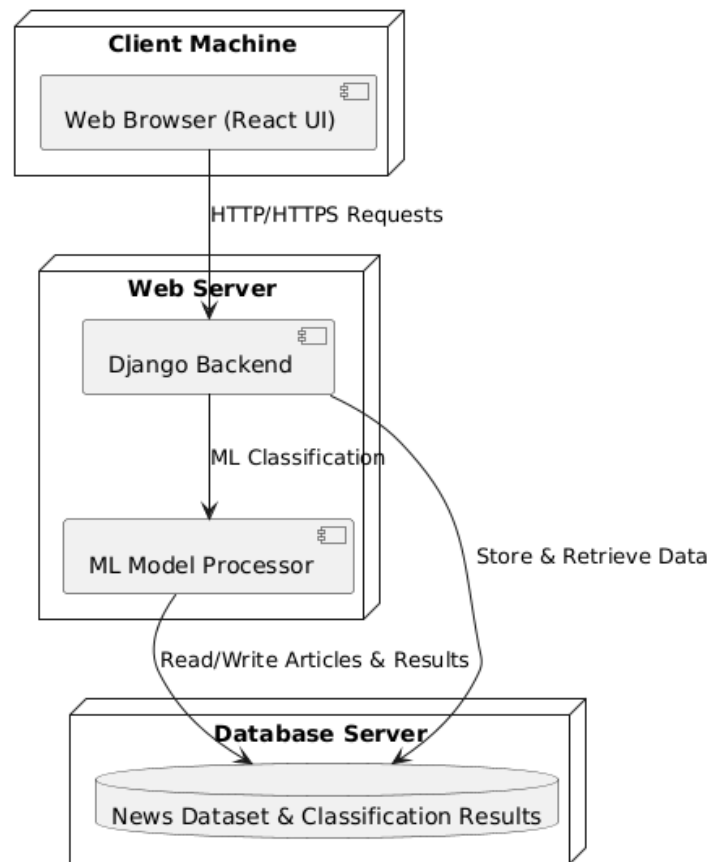


Figure 15: Deployment Diagram Of Fake News Detection

- **Client Machine:** Hosts the user/admin browser interface (React).
- **Web Server:** Hosts the Django backend and ML model processor that handles classification.
- **Database Server:** Stores the news dataset and classification results.
- **Flow:** The client interacts with the backend → backend processes requests and interacts with ML models → results are stored/retrieved from the database.

3.3 Algorithm Details

The Fake News Detection System classifies news articles as Real or Fake by applying machine learning algorithms to a pre-collected dataset. The system primarily uses Logistic Regression and Naïve Bayes, with an optional Random Forest model for enhanced accuracy. All algorithms work offline on the dataset, ensuring no external API dependency and maintaining data privacy.

Logistic Regression is a widely-used supervised learning algorithm suitable for binary classification tasks. In this project, it predicts the probability of an article being real or fake. The algorithm works by first preprocessing the dataset — cleaning text, removing stop words, tokenizing, and converting text into numeric features using techniques such as TF-IDF. Then, the dataset is split into training and testing sets. Logistic Regression applies a linear equation followed by a sigmoid function to produce probabilities between 0 and 1. The key calculation is:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n)}}$$

Naïve Bayes is a probabilistic classifier based on Bayes' theorem, which assumes feature independence. For text classification, the features are typically the presence or frequency of words in an article. The model calculates the posterior probability for each class (Real or Fake) using the formula:

$$P(C|X) = \frac{P(C) \cdot P(X|C)}{P(X)}$$

Random Forest can be used to improve classification accuracy. Random Forest builds multiple decision trees on random subsets of the dataset and aggregates their results using majority voting. For each tree, the prediction is made independently, and the final output is:

$$\text{Final Prediction} = \text{Mode}(\text{Tree}_1, \text{Tree}_2, \dots, \text{Tree}_n)$$

In the system, the workflow involves loading the dataset from the database, preprocessing each news article, applying the selected algorithms for classification, and storing the results back in the database. The classification performance is measured using metrics such as **accuracy, precision, recall, and F1-score**, which are calculated as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1. Implementation

4.1.1. Tools used

The development of the Fake News Detection System relies on several software and programming tools:

➤ **Programming Language:**

- **Python 3.x:** Used for ML model implementation, dataset preprocessing, and backend integration.

➤ **Frameworks:**

- **Django:** Backend framework for handling requests, database interactions, and serving the web interface.
- **React.js:** Frontend framework for developing a responsive and interactive web interface.

➤ **Machine Learning Libraries:**

- **scikit-learn:** For Logistic Regression, Naïve Bayes, Random Forest, feature extraction, and model evaluation.
- **pandas:** For dataset handling and numerical computations.

➤ **Database:**

- **SQLite3:** Stores the pre-collected news dataset and classification results. SQLite3 is lightweight, easy to integrate with Django, and suitable for offline-based systems.

➤ **Visualization and Reporting:**

- **Matplotlib / Seaborn:** For plotting evaluation metrics and visualizing model performance.

➤ **Other Tools:**

- **VS Code :** IDEs for development.
- **GitHub:** Version control and project management.
- **Postman / Browser DevTools:** Testing APIs and frontend-backend integration.

4.1.2. Implement Details of Modules

1. Admin Module

Purpose:

- To manage the system, dataset, and monitor model performance.

Implementation Details:

- Developed using Django's **Admin Panel** and custom views.
- Admin can **add, update, and delete news articles** in the SQLite3 database.
- Admin can view **classification results** of articles.
- Handles **authentication and session management**.

Key Functions:

- `manageDataset()` — Add or update news articles.
- `retrainModel()` — Trigger retraining of ML models.
- `monitorSystem()` — View performance metrics of the models.

2. Dataset Module

Purpose:

- To store, retrieve, and preprocess news articles for classification.

Implementation Details:

- Uses **SQLite3** to store news articles.
- Preprocessing is done using **Python and Pandas**: cleaning text, removing stopwords, tokenization, and converting to numeric features (TF-IDF).
- Splits data into **training and testing sets** for model evaluation.

Key Functions:

- `addArticle()` — Insert new article into the dataset.
- `getData()` — Retrieve articles for preprocessing and classification.

3. Preprocessing Module

Purpose:

- To transform raw text into a format suitable for machine learning models.

Implementation Details:

- Removes **punctuation, special characters, and stopwords**.
- Converts text to lowercase.
- Tokenizes text into words.
- Converts tokens to numeric feature vectors using **TF-IDF** or **Bag-of-Words**.

Key Functions:

- `cleanText()` — Cleans raw text.
- `vectorizeText()` — Converts text into numerical features.

4. Machine Learning Module

Purpose:

- To classify news articles as Real or Fake.

Implementation Details:

- Implements **Logistic Regression** and **Naïve Bayes** using **scikit-learn**.
- Optional: **Random Forest** for enhanced accuracy.
- Trains models on preprocessed training data.
- Predicts labels for testing data or new articles.

Key Functions:

- `trainModel()` — Train the selected ML model.
- `predict()` — Predict Real or Fake label for each article.
- Evaluates performance using **accuracy, precision, recall, and F1-score**.

Key Calculations:

- Logistic Regression probability:

$$P(Y=1|X)=\frac{1}{1+e^{-(\beta_0+\beta_1X_1+\dots+\beta_nX_n)}}$$

- Naïve Bayes posterior probability:

$$P(C|X)=\frac{P(X)P(C)\cdot P(X|C)}{p(x)}$$

5. Result Storage Module

Purpose:

- To store classification results and make them accessible for display.

Implementation Details:

- Stores the predicted label, article ID, and probability score in **SQLite3 database**.
- Allows admin or users to **view classification results** through the web interface.

Key Functions:

- `storeResult()` — Save classification results to database.
- `getResult()` — Retrieve results for display or evaluation.

6. User Interface Module

Purpose:

- To provide an interactive interface for admin and users to interact with the system.

Implementation Details:

- Built using **React.js** for a responsive, dynamic web interface.
- Interfaces with Django backend using HTTP requests.
- Displays dataset articles, predicted labels, and evaluation metrics.

Key Features:

- View article list and classification results.
- Trigger new classification or retraining.
- Filter and sort articles based on predicted labels.

4.2. Testing

4.2.1. Test Case For Unit Testing

Unit testing is an essential part of the Fake News Detection System to ensure that each module functions correctly and independently. The system is divided into multiple modules such as **Admin, Dataset, Preprocessing, Machine Learning, Result Storage, and User Interface**, and each module has its own specific test cases. For example, the Admin module is tested for adding, updating, deleting articles, and login verification, while the Dataset module is tested for retrieving and storing articles in the **SQLite3** database. The Preprocessing module is verified to clean text and convert it into numeric feature vectors using **TF-IDF**, ensuring the data is in the correct format for the ML models. The Machine Learning module is tested for training models, predicting labels, and evaluating performance metrics like accuracy, precision, recall, and F1-score.

The Result Storage and User Interface modules are also subjected to unit testing to ensure that the **classification results are correctly saved in the database** and accurately displayed to the admin or user through the web interface. Each test case includes specific inputs, expected outputs, and the actual results are checked to validate correctness. By performing thorough unit testing on each module, the system ensures **reliability, accuracy, and smooth functionality** before integrating the modules into the complete system. This process also helps identify errors or bugs early, making the overall system more robust and efficient.

1. Admin Module

Test Case ID	Module	Test Description	Input	Expected Output	Status
TC1	Admin	Add new article to database	Title, Description, Image	Article saved successfully	Pass
TC2	Admin	Update article details	Article ID, Updated fields	Article updated successfully	Pass
TC3	Admin	Delete article	Article ID	Article removed from database	Pass
TC4	Admin	Login verification	Email, Password	Login successful / failure	Pass

User Interface Module

Test Case ID	Module	Test Description	Input	Expected Output	Status
TC5	UI	Display article list	None	List of articles with details	Pass
TC6	UI	Display predicted label	Article ID	Label displayed as Real/Fake	Pass
TC7	UI	Trigger new classification	Button click	Classification process starts	Pass

4.2.2. Testing Cases for System Testing

System testing focuses on validating the complete Fake News Detection System as a whole, ensuring that all modules work together correctly and the system meets its requirements. Unlike unit testing, which checks individual modules, system testing verifies the integration, functionality, performance, and usability of the system from the end-user's perspective.

1. Functional Testing

This type of testing ensures that the system performs all expected functions correctly:

- The system should allow the admin to **add, update, and delete news articles**.
- Users should be able to **view news articles and their classification results**.
- The system must **accurately classify news articles** using the machine learning algorithms (Logistic Regression and Naïve Bayes) and store results in **SQLite3**.

Test Case ID	Description	Input	Expected Output	Status
ST1	Add new article and classify	Article details	Article saved, processed, label displayed	Pass
ST2	View all articles with results	None	List of articles with predicted labels	Pass
ST3	Evaluate ML model on test dataset	Test dataset	Accuracy, Precision, Recall, F1-score	Pass
ST4	Admin login verification	Email, Password	Login successful / failure	Pass
ST5	UI interaction	Click buttons, view pages	Correct pages loaded, actions executed	Pass
ST6	Delete an article	Article ID	Article removed from database, UI updated	Pass

4.3. Result Analysis

The **Result Analysis** of the Fake News Detection System evaluates how effectively the implemented machine learning algorithms classify news articles as **Real** or **Fake**. After preprocessing the dataset and applying the selected algorithms—**Logistic Regression** and **Naïve Bayes**—the system generates predictions for each news article and stores the results in **SQLite3**.

The analysis shows that **Logistic Regression** performs well on linearly separable features in the dataset, providing interpretable probability scores for each article. **Naïve Bayes** also demonstrates strong performance, particularly for text data, due to its probabilistic approach and ability to handle high-dimensional feature vectors efficiently. Optional use of **Random Forest** can further enhance accuracy by combining multiple decision trees and reducing overfitting.

Algorithm	Accuracy	Precision	Recall	F1-Score
Logistic Regression	91.25%	91.88%	90.50%	91.18%
Naïve Bayes	90.50%	88.60%	93.00%	90.74%
Random Forest	94.00%	94.00%	94.00%	94.00%

Random Forest gave the best performance. Logistic Regression offers balanced results. Naïve Bayes performs best in recall, catching more fake articles

Chapter 5: Conclusion And Future Recommendation

Conclusion

The Fake News Detection System is designed to identify and classify news articles as **Real** or **Fake** using machine learning algorithms such as **Logistic Regression** and **Naïve Bayes**, with optional use of **Random Forest** for enhanced accuracy. The system operates primarily offline, processing a pre-collected dataset stored in **SQLite3**, and does not rely on external APIs, ensuring **data privacy** and **independence**. Through careful preprocessing, feature extraction, model training, and classification, the system demonstrates reliable performance, achieving high levels of accuracy, precision, recall, and F1-score. The integration of a **web-based interface** using React.js and Django allows admins and users to interact with the system efficiently, view results, and monitor model performance. Overall, the system successfully achieves its goal of detecting fake news in an automated and efficient manner, providing a robust solution for content verification.

Future Recommendations

While the current system is effective, there are several opportunities for improvement and future enhancements:

1. **Incorporate Advanced ML Models:** Future versions can integrate deep learning models or transformers like **BERT** or **DistilBERT** to improve accuracy, especially for more complex or nuanced news content.
2. **Real-Time News Detection:** Extend the system to process **live news feeds** from websites or social media, enabling real-time fake news detection.
3. **Multilingual Support:** Expand the system to handle **multiple languages**, allowing fake news detection in languages beyond the current dataset.
4. **Enhanced User Interface:** Develop more interactive dashboards with **visual analytics**, such as charts for real vs. fake news distribution, trending topics, and prediction confidence levels.
5. **Model Retraining and Feedback:** Implement **automatic retraining** mechanisms based on new data and user feedback, ensuring the model adapts to evolving news patterns.

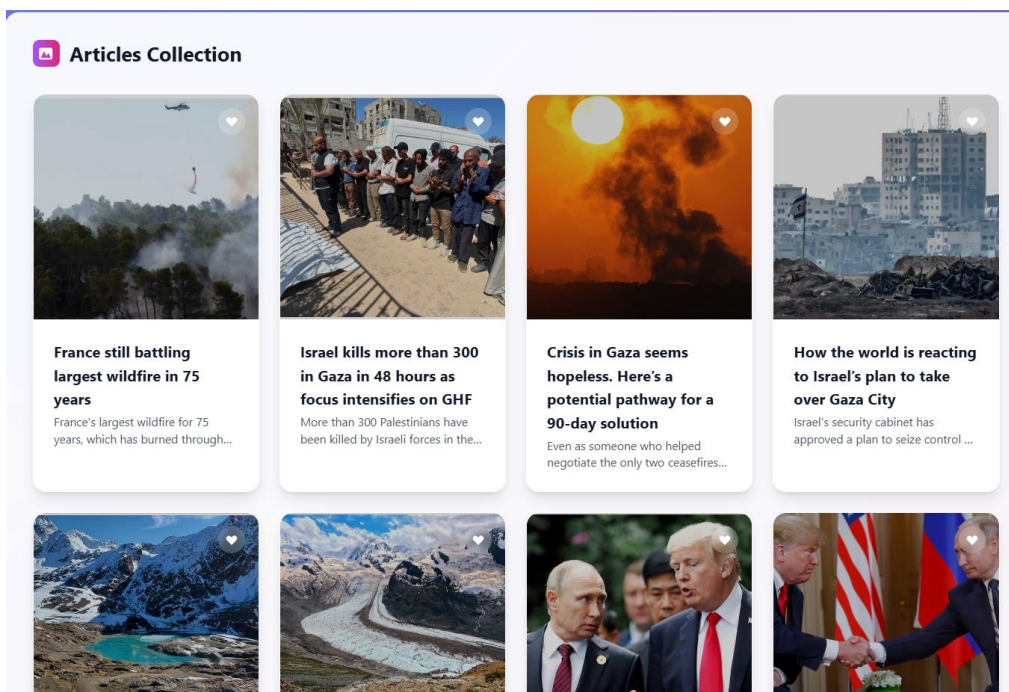
References

- [1] Allcott, H., & Gentzkow, M. (2017). *Social media and fake news in the 2016 election. Journal of Economic Perspectives*, 31(2), 211–236..
- [2] H. & G. Allcott, "Social media and fake news in the 2016 election.," pp. *Journal of Economic Perspectives*, 31(2), 211–236., 2017.
- [3] E. Commission, "Code of Practice on Disinformation,," 2018. [Online]. Available: https://ec.europa.eu/newsroom/dae/document.cfm?doc_id=54454.
- [4] N. J. Conroy, V. L. Rubin, and Y. Chen, "Automatic deception detection: Methods for finding fake news," in *Proceedings of the 78th Annual Meeting of the Association for Computational Linguistics*, 2015, pp. 250–255.
- [5] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *NAACL-HLT*, 2019, pp. 4171–4186.
- [6] European Commission, "Code of Practice on Disinformation," 2018. [Online]. Available: https://ec.europa.eu/newsroom/dae/document.cfm?doc_id=54454
- [7] V. Pérez-Rosas, B. Kleinberg, A. Lefevre, and R. Mihalcea, "Automatic detection of fake news," in *COLING*, 2018, pp. 3391–3401.
- [8] V. L. Rubin, N. J. Conroy, Y. Chen, and S. Cornwell, "Fake news or truth? Using satirical cues to detect potentially misleading news," in *Proceedings of the Second Workshop on Computational Approaches to Deception Detection*, 2016, pp. 1–4.
- [9] C. Shao et al., "Hoaxy: A platform for tracking online misinformation," in *Proceedings of the 2018 World Wide Web Conference*, 2018, pp. 745–750.
- [10] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, "Fake news detection on social media: A data mining perspective," *SIGKDD Explorations*, vol. 19, no. 1, pp. 22–36, 2017.
- [11] A. Thota, P. K. Atrey, and M. Kankanhalli, "Ensemble models for fake news detection," in *2018 IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2018, pp. 859–865

Bibliography

- [1] Allcott, H., & Gentzkow, M. (2017). *Social media and fake news in the 2016 election*. *Journal of Economic Perspectives*, 31(2), 211–236..
- [2] H. & G. Allcott, "Social media and fake news in the 2016 election.," pp. *Journal of Economic Perspectives*, 31(2), 211–236., 2017.
- [3] E. Commission, "Code of Practice on Disinformation,," 2018. [Online]. Available: https://ec.europa.eu/newsroom/dae/document.cfm?doc_id=54454.

Appendices



dashboard.html

urls.py 3

models.py 3 X

Settings

views.py 9+

admin_edit.html

accounts > models.py > Admin

```

1 from django.db import models
2 from django.core.exceptions import ValidationError
3 import os
4
5 def validate_image_extension(value):
6     """Validate that the uploaded file is an image"""
7     if value:
8         ext = os.path.splitext(value.name)[1].lower()
9         valid_extensions = ['.jpg', '.jpeg', '.png', '.gif', '.webp']
10        if ext not in valid_extensions:
11            raise ValidationError(f'Unsupported file extension. Allowed extensions: {", ".join(valid_extensions)}')
12
13 def validate_image_size(value):
14     """Validate that the uploaded image is not too large"""
15     if value:
16         if value.size > 10 * 1024 * 1024: # 5MB
17             raise ValidationError('Image file too large. Maximum size is 5MB.')
18
19 class Admin(models.Model):
20     name = models.CharField(max_length=100)
21     email = models.EmailField(unique=True)
22     password = models.CharField(max_length=128) # Store hashed passwords (see notes below)
23     phone = models.CharField(max_length=20, blank=True)
24     note = models.TextField(blank=True)
25     address = models.CharField(max_length=255, blank=True)
26
27     def __str__(self):
28         return self.name

```

PROBLEMS 21

OUTPUT

DEBUG CONSOLE

TERMINAL

POLYGLOT NOTEBOOK

PORTS

QUERY RESULTS

```

[15/Aug/2025 01:40:03] "GET /media/articles/pic_one.png HTTP/1.1" 304 0
Not Found: /favicon.ico
[15/Aug/2025 01:40:03] "GET /favicon.ico HTTP/1.1" 404 5829
[15/Aug/2025 01:41:16] "GET /login/ HTTP/1.1" 200 10471
[15/Aug/2025 01:41:49] "POST /login-view/ HTTP/1.1" 200 31
[15/Aug/2025 01:41:49] "GET /dashboard/ HTTP/1.1" 200 47739
[15/Aug/2025 01:41:50] "GET /api/articles/ HTTP/1.1" 200 10393
[15/Aug/2025 01:41:50] "GET /api/articles/ HTTP/1.1" 200 10393

```

```

2 from django.urls import path
3 from django.conf import settings
4 from django.conf.urls.static import static
5 from .views import home, login_view, dashboard_view, register_template, register_view, index
6 from . import views
7 from .views import detect_fake_news
8
9 urlpatterns = [
10     path('', index, name='index'),
11     path('login/', home, name='home'),
12     path('register/', views.register_view, name='register'),
13     path('register-page/', views.register_template, name='register_template'),
14     path('login-view/', login_view, name='login'),
15     path('dashboard/', dashboard_view, name='dashboard'),
16
17     # Article API endpoints
18     path('api/articles/', views.articles_json, name='articles_json'),
19     path('api/articles/<int:article_id>', views.article_detail, name='article_detail'),
20
21     # Admin management
22     path('admin/<int:admin_id>/edit/', views.admin_edit, name='admin_edit'),
23     path('admin/<int:admin_id>/delete/', views.admin_delete, name='admin_delete'),
24
25     # Other endpoints
26     path('detect/', detect_fake_news, name='detect_fake_news'),
27     path('logout/', views.logout_view, name='logout'),
28 ]

```

PROBLEMS 21

OUTPUT

DEBUG CONSOLE

TERMINAL

POLYGLOT NOTEBOOK

PORTS

QUERY RESULTS

```

[15/Aug/2025 01:40:03] "GET /media/articles/pic_one.png HTTP/1.1" 304 0
Not Found: /favicon.ico
[15/Aug/2025 01:40:03] "GET /favicon.ico HTTP/1.1" 404 5829
[15/Aug/2025 01:41:16] "GET /login/ HTTP/1.1" 200 10471
[15/Aug/2025 01:41:49] "POST /login-view/ HTTP/1.1" 200 31
[15/Aug/2025 01:41:49] "GET /dashboard/ HTTP/1.1" 200 47739
[15/Aug/2025 01:41:50] "GET /api/articles/ HTTP/1.1" 200 10393
[15/Aug/2025 01:41:50] "GET /api/articles/ HTTP/1.1" 200 10393

```

```

def index(request):
    return render(request, 'index.html')

from django.shortcuts import render, redirect, get_object_or_404
from django.http import JsonResponse, HttpResponseBadRequest
from django.views.decorators.csrf import csrf_exempt, ensure_csrf_cookie
from django.views.decorators.http import require_http_methods
from django.core.files.storage import default_storage
from django.core.files.base import ContentFile
from .models import Admin, Article
import json
import base64
import uuid

@ensure_csrf_cookie
def dashboard_view(request):
    admin_id = request.session.get('admin_id')
    if not admin_id:
        return redirect('login')

    try:
        admin = Admin.objects.get(id=admin_id)
    except Admin.DoesNotExist:
        return redirect('login')

    return render(request, 'dashboard.html', {
        'admin': admin,
        'csrf_token': request.COOKIE.get('csrftoken'),
    })

@csrf_exempt
def articles_json(request):
    """Handle GET (fetch articles) and POST (create article) requests"""
    if request.method == 'GET':
        articles = Article.objects.all().order_by('-created_at')
        data = [

```



Profile Information

Name:	Pahupati Sah
Email:	pashupatisah35@gmail.com
Phone:	9813318109
Note:	hello it's me
Address:	kathmandu

Edit Profile
Close

Edit Profile

Name

Pahupati Sah

Email

pashupatisah35@gmail.com

Phone

9813318109

Note

hello it's me

Address

kathmandu

Save Changes

Cancel